Compilers midterm: all 29 methods + gotchas on one page

blue = method, red ! = gotcha. Numbers = sections of *Methods, Practice & Gotchas*.

Lexing

1 Which phase catches the error?

Ask in order, stop at the first "no": **Lexical** (can the characters be cut into legal tokens by longest match? bad character, unclosed string/comment) ▹ **Syntax** (is the *token sequence* allowed by the grammar?) ▹ **Semantic** (types, declarations, number of arguments) ▹ else run-time/logic.

! "Looks like a bad identifier" is not lexical: `int 3x` cuts into NUM(3) ID(x), two legal tokens; the grammar rejects them, so it is a *syntax* error. The lexer never knows what the grammar expects.

2 Regular expression for a token

Cut the lexeme into parts; each is mandatory, optional `?`, or repeated `*`/`+`. One sub-expression per part, | inside a part. Test every must-match and must-not example. Precedence: $*$ > concatenation > |.

! A string body written as "anything", `"(x | " | \)*"`, accepts `"x"x"`; with longest match it swallows two strings and the code between them. The body must exclude the closing quote.

3 Thompson: regex → NFA

Inside out; each fragment has one start and one final state. a: $i \xrightarrow{a} f$. Union: new start $\varepsilon \to$ both starts, both finals $\varepsilon \to$ new final. Star: new i, f; ε: $i \to s_0$, $s_f \to s_0$ (again), $s_f \to f$, $i \to f$ (zero times). Concat **B** (textbook): final of $N(s)$ *is* start of $N(t)$; **A**: an ε-edge. Count in the parse tree: n_ℓ leaves, $n_|$, n_*, $n_.$ (`abc` has 2); rewrite $r? = (r \mid \varepsilon)$, $r^+ = rr^*$ first.
States A = $2(n_\ell+n_|+n_*)$, B = A − $n_.$. **Edges** B = $n_\ell + 4(n_|+n_*)$, A = B + $n_.$.

! State your convention (they differ by $n_.$). $r^+ \to rr^*$ makes its leaves count twice. The star needs the back edge *and* the bypass $i \to f$; no bypass builds r^+.

4 Simulating an NFA

$S = \mathrm{cl}(\{\text{start}\})$; for each symbol c: $S = \mathrm{cl}(\mathrm{move}(S, c))$. Accept iff S holds a final state; $S = \emptyset$ means reject at once.

! Take the ε-closure after *every* move, not only at the start.

5 Subset construction: NFA → DFA

Start $A = \mathrm{cl}(\{s_0\})$. For each new set S and symbol c: $T = \mathrm{cl}(\mathrm{move}(S, c))$, name it if new. Stop when nothing new. Accepting iff the set contains ≥ 1 accepting NFA state.

! Sets are unordered: $\{0,1\} = \{1,0\}$; naming them twice makes a fake state. $\emptyset$ is a dead state.

6 DFA minimisation (table filling)

Add a dead state if transitions are missing. List all pairs; mark (accepting, non-accepting). Sweep: mark (p, q) if some symbol sends it to a marked pair. Repeat until a sweep marks nothing. Merge unmarked pairs.

! Redirect *every* edge into a merged state (self-loops too), and stop only after a sweep that marks nothing.

7 State elimination: DFA → regex

New start $s \xrightarrow{\varepsilon}$ old start (if it has incoming edges); new final f with ε from every accepting state. Remove q with self-loop L: each $p \xrightarrow{X} q \xrightarrow{Y} r$ gives $p \xrightarrow{XL^*Y} r$, joined with | to an existing $p \to r$ ($p = r$: self-loop). Answer: label of $s \to f$.

! No new start when the old one has incoming edges loses the paths that leave and re-enter it.

8 Brzozowski derivatives

$\partial_a a = \varepsilon$, $\partial_a b = \partial_a \varepsilon = \partial_a \emptyset = \emptyset$, $\partial_a(R \mid S) = \partial_a R \mid \partial_a S$, $\partial_a(R^*) = (\partial_a R)R^*$, $\partial_a(RS) = (\partial_a R)S \mid \nu(R)\,\partial_a S$, where $\nu(R) = \varepsilon$ if R is nullable, else $\emptyset$. Accept w iff $\partial_w R$ is nullable; the distinct derivatives are the DFA states.

! Dropping $\nu(R)\,\partial_a S$ when R can be empty (e.g. x^*) rejects `a` for x^*a.

9 Designing a DFA from a description

Decide what to *remember* (a remainder, how much of a pattern has been seen, inside/outside a comment). One state per value; start = value for ε; transition = update rule; mark accepting values. Test 2 accepted and 2 rejected strings.

! Binary mod 3: reading bit b *doubles*: $r \mapsto (2r + b) \bmod 3$, not $(r + b) \bmod 3$.

10 Not regular; DFA lower bounds

u, v are *distinguishable* if one suffix z puts exactly one of uz, vz in L, so they need different states. Infinitely many pairwise distinguishable ⇒ not regular; k of them ⇒ ≥ k states. Template: $u = a^i$, $v = a^j$, $i < j$, $z = b^i$: $uz \in L$, $vz \notin L$.

! Exponents do not mean non-regular ($a^n b^m$ is regular). It takes an unbounded *link* between counts.

11 Combined scanner DFA and rule priority

Pick character classes; build one DFA for all rules (NFA per rule + subset construction); label each accepting state with the **first-listed** rule accepting there. A rule that labels no state never fires.

! Priority breaks only *ties in length*: with keyword `in` listed first, `inx` is still an identifier (longer match).

12 Longest match with backup, counting reads

Read while a transition exists; remember the last accepting state and position. Stuck or EOF: back up to it, emit, restart there; characters beyond it are re-read (pushback). No accepting state seen: error on one character, skip it. Reads = every character fetched, including the one that got you stuck and all re-reads (rules a, a^*b on `aaaa`: 4+3+2+1 = 10).

! Do not stop at the first accepting state (`abb` by rule 2 continues to `abbb` by rule 3). Stuck in a non-accepting state: back up to the last *accepting* one.

13 Lexer states for nested comments

Nested comments are not regular: switch to a lexer state (flex start condition) with its own rules and a depth counter: `/*` → depth++, `*/` → depth−−, back to normal at 0. Add an end-of-file rule inside.

! Without `<<EOF>>`, an unclosed comment silently swallows the rest of the file instead of being a lexical error.

Grammars

14 Writing a CFG for a language

Equal counts only come from producing the two symbols *in the same step* ($S \to aSb$). (1) Write the strings as one-letter blocks, split sums: $a^n b^m c^{n+m} = a^n\,b^m\,c^m\,c^n$. (2) Draw an arc between blocks with equal counts. Arcs must *nest* like parentheses; crossing arcs ($a^n b^m c^n d^m$) mean no CFG. (3) One nonterminal per arc, outside in: wrap + exit $S \to aSc \mid B$, $B \to bBc \mid \varepsilon$. (4) Unlinked block $A \to aA \mid \varepsilon$; blocks side by side $S \to XY$. (5) Derive the smallest strings. Toolbox: $a^n b^{2n}$: $S \to aSbb \mid \varepsilon$; $a^n b^n c^m$: $S \to XC$; $L_1 \cup L_2$: $S \to S_1 \mid S_2$; $\#a = \#b$: $S \to aSbS \mid bSaS \mid \varepsilon$.

! $S \to aS \mid Sb \mid \varepsilon$ generates a^*b^*: a and b are added in *different* steps, so nothing links their counts.

15 Leftmost/rightmost derivations, parse trees

Build the parse tree first (choose each production by first/last symbols and length). Leftmost = depth-first walk, left child first: each interior node is one step replacing the leftmost nonterminal. Rightmost = right child first. Check: steps = interior nodes; write the whole string at every step.

! The rightmost derivation builds the *right* end of the sentence first, so its middle strings look nothing like the leftmost ones; both give the same tree.

16 Ambiguity, precedence, associativity

Prove: take the shortest sentence with 2 operators (`id+id*id`, or `id+id+id` for one operator) and draw its two trees: one groups the left pair, the other the right pair. Two trees = ambiguous. **Fix, 3 steps:** (1) list the levels loosest → tightest, one nonterminal each, atoms last: E (+, −) ⊃ T ($*$, /) ⊃ U (unary −) ⊃ P. (2) write each level with X = this level, Y = the next tighter one: left-assoc $X \to X\,op\,Y \mid Y$; right-assoc $X \to Y\,op\,X \mid Y$; non-assoc $X \to Y\,op\,Y \mid Y$; prefix unary $X \to op\,X \mid Y$. (3) atoms $P \to (E) \mid$ `id`: parentheses jump back to the top. Result: $E \to E + T \mid T$, $T \to T * U \mid U$, $U \to -U \mid P$, $P \to (E) \mid$ `id`; a right-assoc `^` gets its own level $F \to U \,\hat{}\, F \mid U$ between T and U. **Check:** tightest operator lowest in the tree of `id+id*id`; the recursion side is the associativity; same language, one tree.

! A leftmost plus a rightmost derivation proves nothing: two *trees* = two *leftmost* derivations. $X \to X\,op\,X \mid Y$ keeps the ambiguity. $P \to (P)$ instead of (E) loses `(a+b)*c`.

LL(1) parsing

17 Recursive descent: grammar ↔ code

Grammar → code. One procedure per nonterminal, global lookahead `la`; `match(t)`: `la==t`? advance : error. SEL($A \to \alpha$) = FIRST(α) \ ε, ∪FOLLOW(A) if α nullable. `void A(){ if (la in SEL1) {body1} else if (la in SEL2) {body2} else error(); }` Body left to right: terminal t → `match(t)`, nonterminal B → `B()`, ε → nothing. $A' \to t\beta A' \mid \varepsilon$ may be coded `while (la==t){match(t); β}`. Driver: `la=next(); S(); if (la!=EOF) error();` **Code → grammar.** Procedure = nonterminal; start = what the driver calls. Each `if` branch = one production: its `match`/call sequence in order = right side, its condition = SEL. Empty branch = $A \to \varepsilon$, its tokens are FOLLOW(A). `while (la==t){..}` = new $A' \to t \ldots A' \mid \varepsilon$ at that spot. Conditions pairwise disjoint in every procedure ⟺ LL(1). Trace: indent per call, note the deciding token; only `match` consumes.

! $A \to A\alpha$ becomes `A(){A();...}`: same lookahead, nothing consumed, stack overflow. A condition token is in the right side only if `match` follows; matching a FOLLOW token in the callee steals it from the caller: `(id)` rejected. `else{}` without a FOLLOW test defers the check: no `la==EOF` in the driver and `id id` is accepted.

18 Nullable

Pass 1: A with $A \to \varepsilon$. Each later pass, in order: mark A if a right side is *all* marked nonterminals (marks from earlier in the same pass count; a terminal never qualifies). Stop after a pass with nothing new. A string is nullable iff every symbol is.

! One pass is not enough: nullability climbs chains ($D \to C$, $C \to A$, $A \to \varepsilon$).

19 FIRST

FIRST(t) = {t}; $X \to \varepsilon$ puts ε. For $X \to Y_1 \cdots Y_k$: add FIRST(Y_1) \ ε; continue to Y_2 while nullable; stop at the first non-nullable; all nullable ⇒ add ε. Iterate to a fixed point.

! $\varepsilon \in$ FIRST(X) only if a *whole* right side vanishes: $T \to FT'$ with F not nullable gives $\varepsilon \notin$ FIRST(T).

20 FOLLOW

\$ ∈ FOLLOW(S). For every *occurrence* $B \to \alpha A \beta$: FIRST(β) \ ε ⊆ FOLLOW(A); if β is nullable or empty, FOLLOW(B) ⊆ FOLLOW(A). Iterate. Never contains ε.

! It comes from where A *occurs*, not from A's own productions. Common miss: C inside another nonterminal's rule $B \to C$.

21 LL(1) table and conflicts

Number the productions. $A \to \alpha$ goes in $M[A, a]$ for $a \in$ FIRST(α); if α is nullable, also in $M[A, b]$ for $b \in$ FOLLOW(A) (\$ included). Two numbers in one cell: FIRST/FIRST or FIRST/FOLLOW conflict.

! Use FIRST of the *right side*, not of the nonterminal. There is no ε column: ε-productions go under FOLLOW.

22 LL(1) stack trace

Columns: stack (top on the right) | input | action; start \$$S$ | w\$. Top X, lookahead a: $X = a$ = \$ accept. Terminal: $X = a$ match (pop, advance), else error. Nonterminal: $M[X, a] = X \to Y_1 \cdots Y_k$: pop, push $Y_k \ldots Y_1$ (Y_1 on top); ε: just pop; empty cell: error. Rows = productions used + tokens matched +1.

! Only a match shortens the input. Push reversed: $S \to (L)$ gives `)` `L` `(` with `(` on top.

23 Where is the syntax error caught?

Table: run the trace and report the top X, the lookahead a, and why (empty $M[X, a]$, or a terminal mismatch). Code: simulate the calls with the conditions *as written*, and report the procedure and its caller. The code can be *later*: an ε branch written as a default accepts any token; a one-alternative procedure checks nothing before calling.

! Both parsers reject exactly the same inputs; only *where* differs. The table is never later than the code.

Transformations & decisions

24 Immediate left recursion

$A \to A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n$ ⇒ $A \to \beta_1 A' \mid \cdots \mid \beta_n A'$, $A' \to \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon$. No new nonterminal allowed and $L \to LX \mid \varepsilon$: $L \to XL \mid \varepsilon$.

! Every β needs the A' suffix, and A' needs its ε; forgetting either changes the language.

25 Indirect left recursion

Order $A_1..A_n$. For each i, for $j < i$: replace $A_i \to A_j\gamma$ by $A_i \to \delta\gamma$ for every $A_j \to \delta$; then remove A_i's immediate left recursion. Rebuild FIRST/FOLLOW and the table.

! Necessary, not sufficient, for LL(1), and the order matters: $A \to Ba \mid c$, $B \to Ab \mid d$: order A, B is not LL(1); order B, A is.

26 Left factoring

Longest common prefix α: $A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma$ ⇒ $A \to \alpha A' \mid \gamma$, $A' \to \beta_1 \mid \beta_2$. Repeat until no two alternatives share a first symbol.

! It removes prefixes, not ambiguity: if the conflict survives (dangling else), look for two trees.

27 Eliminating ε-productions

Find the nullables. A right side with k nullable occurrences gets 2^k keep/delete versions; drop the empty one, duplicates and $A \to A$. Delete every ε-production. If S was nullable, add $S' \to S \mid \varepsilon$. Example: $A \to BaB$, $B \to b \mid \varepsilon$ ⇒ $A \to BaB \mid aB \mid Ba \mid a$, $B \to b$.

! 2^k minus the empty one: the mock's $(S)S$ gives four. Forgetting $S' \to S \mid \varepsilon$ loses ε from the language.

28 Is this grammar LL(1)?

In order; any "yes" means not LL(1): (1) ambiguous? (2) left-recursive, directly or indirectly? (3) two alternatives with overlapping FIRST (common prefix)? (4) a nullable alternative of A whose FOLLOW(A) meets another alternative's FIRST? Justify with the cell and the two productions.

! It describes the *grammar*, not the language: left recursion and prefixes can be fixed; ambiguity cannot by these transformations.

29 Chomsky type of a grammar and of its language

3: $A \to aB \mid a \mid \varepsilon$ (finite automaton, $(a|b)^*a$). 2: $A \to \alpha$ (pushdown automaton, $a^n b^n$). 1: $\alpha \to \beta$, $|\alpha| \le |\beta|$ (linear-bounded automaton, $a^n b^n c^n$). 0: $\alpha \to \beta$ (Turing machine). Grammar type = the most restrictive row *all* productions satisfy. Language type: if a regex/DFA exists, 3; else prove it non-regular (§10) and give a CFG, 2.

! The type of the given grammar is not the type of its language: a context-free grammar can generate a regular language.